

Programação Recursiva: processamento de listas em Lisp

Orlando de A Figueiredo

IGCE/Unesp

15 de Agosto de 2017

Programação recursiva

- ▶ A programação recursiva é naturalmente declarativa.

Processamento de listas

- ▶ O processamento de listas é naturalmente recursivo.
- ▶ Processar uma lista é processar sua cauda (sublista obtida por intermédio de `cdr`) primeiro.
- ▶ A lista vazia torna-se a condição de parada da recursão (em geral).

Anatomia de uma função recursiva de processamento de listas

- ▶ A função começa verificando se a lista é vazia.
- ▶ Se for, resulta no caso base.
- ▶ Se não for, tem-se o caso geral, que deve envolver uma chamada recursiva na cauda da lista, e o exame da cabeça da lista (car).

```
(define (<função> l [etc])  
  (if (null? l)  
      (... retorno do caso base ...) ;then  
      (... retorno do caso geral ...)); else
```

Mais de perto

- ▶ Observe a chamada recursiva da cauda.
- ▶ (Obs.: dependendo do problema, o caso base pode envolver também esses elementos.)

```
(define (<função> l [etc])  
  (if (null? l)  
      (... caso base ...)  
      (... caso geral  
        ..  
        .. (car l) ..  
        ..  
        .. (<função> (cdr l) [etc]) ..  
        ..  
        ..)))
```

Outra forma de ver: pensando como a máquina

- ▶ O ser humano, quando vê uma lista, emprega seus superpoderes (visão simultânea do todo, paralelismo, etc.)
- ▶ Uma máquina é muito mais limitada.
- ▶ Digamos que a máquina só consegue enxergar claramente a cabeça da lista.
- ▶ A cauda da lista é algo difuso e indiscernível, mas que se resolve na recursão.

Humano: '(4 0 -2 3 9 6 20 -4 7)

Máquina: '(4 <bla bla bla bla..>)

Resumo

- ▶ Concentre-se em testar alguma propriedade da cabeça da lista.
- ▶ Combine-a, se necessário, com a chamada recursiva sobre a cauda, como se fosse algo pronto.
- ▶ Imagem: pense que você tem à sua disposição sobre a mesa vários tijolinhos que devem ser combinados. Uns são do tipo (car 1) (podem ser várias cópias), outros são do tipo (<função> (cdr 1) [etc]).

Exemplo: a função-predicado `pertence?`

- ▶ Parâmetros: `x` (elemento); `l` (lista=conjunto)
- ▶ Retorna `#t` se `x` está na lista `l`

```
(pertence? 3 '(3 1 4))  
>>> #t  
(pertence? 3 '(0 3 1 4))  
>>> #t  
(pertence? 3 '(5 1 4))  
>>> #f
```

Construindo a função `pertence?` passo a passo

- ▶ O foco agora é a cabeça da lista: se ela for igual a `x`, sabemos que é verdade que `x` pertence à lista; o resultado é `#t`.

```
(define (pertence? x l)
  (if (null? l)
      (... base ...)
      (if (equal? x (car l))
          #t
          (... )))))
```

- ▶ Se a cabeça da lista não for x, é hora de analisar a recursão sobre a cauda.
- ▶ Se a recursão sobre a cauda for verdade, o resultado é verdade; se for falso, é falso; logo, o resultado é a própria chamada recursiva.

```
(define (pertence? x l)
  (if (null? l)
      (... base ...)
      (if (equal? x (car l))
          #t
          (pertence? x (cdr l)))))
```

Por fim, o caso base.

- Um conjunto vazio não possui elementos.

```
(define (pertence? x l)
  (if (null? l)
      #f
      (if (equal? x (car l))
          #t
          (pertence? x (cdr l)))))
```

Outro exemplo: a função `comprm` (comprimento da lista)

- ▶ Parâmetro: `l` (lista)

```
(comprm '(3 1 4))  
>>> 3  
(comprm '())  
>>> 0  
(comprm '(5 1 4 6 10))  
>>> 5
```

- ▶ O caso base é trivial.
- ▶ No caso geral, tendo o comprimento da cauda como ponto de partida, basta incrementá-lo.
- ▶ Nesse raro caso, tanto faz o valor dos elementos (a cabeça não é examinada).

```
(define (comprm l)
  (if (null? l)
      0
      (add1 (comprm (cdr l)))))
```

Observação!

- ▶ Tanto no caso de `comprm` quanto dos próximos exercícios, o resultado é um número.
- ▶ Os operadores envolvidos são aritméticos (`add1`, `+`, `-`, etc.) (assim como os operadores do resultado booleano eram booleanos).
- ▶ O caso mais importante é quando o resultado é uma lista (o operador é o `cons`).

```
(define (comprm l)
  (if (null? l)
      0
      (add1 (comprm (cdr l)))))
```

Exercícios – Parte 1

- ▶ Contagem de ocorrências de um valor numa lista.
- ▶ Soma dos elementos de uma lista.
- ▶ Máximo valor de uma lista.

```
(contar 3 '(4 3 8))  
>>> 1  
(contar 3 '(3 5 3 1))  
>>> 2  
(contar 3 '(4 6 0 2))  
>>> 0  
(contar 3 '())  
>>> 0  
(soma '(4 6 0 2))  
>>> 12  
(soma '())  
>>> 0  
(máximo '(1 7 4 12 -8 3))  
>>> 12  
(máximo '())  
>>> -1000
```

Exemplo: a função dobrados

- ▶ Parâmetros: 1 (lista)
- ▶ Retorna **uma nova lista** na qual cada elemento é o dobro do respectivo elemento da lista original.

```
(dobrados '(3 1 4))  
>>> '(6 2 4)  
(dobrados '())  
>>> '()
```

Construindo a função dobrados passo a passo

- ▶ O caso base é trivial. Observe que neste caso o resultado é uma lista, portanto a lista vazia.
- ▶ Considere que a recursão já esteja pronta. Portanto, ela é uma lista correspondente a todos os elementos da cauda dobrados.
- ▶ Como sempre, agora é a vez da cabeça da lista, ela deve ser dobrada e inserida no começo da lista pronta (recursão).

```
(define (dobrados l)
  (if (null? l)
      '()
      (cons
        (* (car l) 2)
        (dobrados (cdr l))))))
```

Exercícios – Parte 2

- ▶ Remover de uma lista a primeira ocorrência de um valor dado.
- ▶ Remover de uma lista todas as ocorrências de um valor dado.

```
(remover-prim 3 '(4 3 8 10 3 5 5 3 2))  
>>> '(4 8 10 3 5 5 3 2)  
(remover-prim 7 '(4 3 8 10 3 5 5 3 2))  
>>> '(4 3 8 10 3 5 5 3 2)  
(remover-prim 3 '())  
>>> '()  
(remover-todos 3 '(4 3 8 10 3 5 5 3 2))  
>>> '(4 8 10 5 5 2)
```

Funções anônimas com `lambda`

Uma pausa para falar de Lisp

- ▶ Funções anônimas em Lisp podem ser definidas pela *special form* `lambda`.
- ▶ Ela consiste numa lista com os parâmetros e o corpo.
- ▶ A variante de `define` para funções é, na verdade, mais um *syntactic sugar* na linguagem.

```
(define (f x y) (qq x (zz y x)))  
(define f (lambda (x y) (qq x (zz y x))))
```

lambda é mais do que isso: funções de mais alta ordem

Uma pausa para falar de Lisp

- ▶ Em Lisp, funções também são *cidadãs de primeira classe*: elas têm o direito de serem valores de variáveis, entradas de arrays, parâmetros e retornos de outras funções, assim como os números, as strings, etc.
- ▶ Funções que recebem ou retornam funções são chamadas *funções de ordem mais alta*

```
(map (lambda (x) (add1 x)) '(10 20 30))  
(map add1 '(10 20 30))
```

Exemplo: a função existe? como uma generalização de pertence?

- ▶ Em Lisp, funções também são *cidadãs de primeira classe*: elas têm o direito de serem valores de variáveis, entradas de arrays, parâmetros e retornos de outras funções, assim como os números, as strings, etc.
- ▶ Funções que recebem ou retornam funções são chamadas *funções de ordem mais alta*

```
(existe? (lambda (x) (equal? x 3)) '(10 20 30))  
(existe? even? '(1 3 7 -1))  
(existe? negative? '(1 3 7 -1))  
(existe?  
  (lambda (x) (and (number? x) (odd? x)))  
  '(#t 5 '(3 a)))
```

Mais exemplos da aplicação de existe?

```
(existe? (lambda (x) (= 0 (modulo x 2))) '(1 3 11))  
>>> #f  
(existe? (lambda (x) (= 0 (modulo x 2))) '(1 2))  
>>> #t  
(existe? number?>('('oi'')))  
>>> #f  
(existe? boolean? '(4 (5 6) #t 5))  
>>> #t
```

Implementando existe?

- ▶ Observe a presença do parâmetro *p* (de *predicado*).
- ▶ Observe o uso de *p!!!* Lembre-se de que ele é uma função, e funções são usadas no começo de uma expressão simbólica.

```
(define (existe? p l)
  (if (null? l)
      #f
      (if (p (car l))
          #t
          (existe? p (cdr l)))))
```

DICA!!! O tipo de retorno booleano

- ▶ Funções que retornam #t ou #f, ou seja, predicados (indicados por convenção pela interrogação) podem ser simplificadas.
- ▶ Geralmente, podem ser escritas como expressões lógicas (combinações de or, and, not, etc.).

```
(define (pertence? x l)  
  (or (equal? x (car l)) (pertence? x (cdr l))))
```

Interpretação declarativa: ou x é a cabeça da lista ou pertence à cauda.

Exercícios – Parte 3

- ▶ Todos os elementos satisfazem predicado.

```
(todos? pair? '((9 12 20) (33) (40 0 1 1)))  
>>> #t  
(todos? number? '(4 5 6 #f))  
>>> #f  
(todos? positive? '(1 2 3))  
>>> #t
```

DICA! Use cond em vez de ifs aninhados

- ▶ Lembre-se do cond
- ▶ Ele é mais sucinto auando há muitos condicionais aninhados.
- ▶ É uma opção pessoal usar um ou outro estilo.

```
(define (pertence? x l)
  (cond [(null? l) #f]
        [(equal? x (car l)) #t]
        [else (pertence? x (cdr l))]))
```

Pensando imperativamente e declarativamente

A mesma expressão (função) pode ser lida de 2 maneiras diferentes:

- ▶ Imperativamente: “chame a recursão, obtenha o resultado, faça operações com ele, etc.”
- ▶ Declarativamente: “a função é isso se a lista é vazia, ou é aquilo se a lista não é vazia (aquilo é uma fórmula envolvendo a recursão, como se ela fosse algo já pronto)”

De volta ao caso base.

- ▶ Pensando imperativamente: se o processamento chegou ao caso da lista vazia, nenhum elemento da lista inicial era igual a x (pois, senão, a função já teria retornado no momento em que uma igualdade fosse detectada).
- ▶ Pensando declarativamente: um conjunto vazio não possui elementos.

```
(define (pertence? x l)
  (if (null? l)
      #f
      (if (equal? x (car l))
          #t
          (pertence? x (cdr l)))))
```

Exercícios – Parte 4

- ▶ Mapear uma função sobre uma lista, isto é, aplicar a função em todos os elementos obtendo uma nova lista.
- ▶ Mapear está para dobrados assim como existe? está para pertence?
- ▶ (Assim como filtrar está para remover-todos)

```
(mapear add1 '(4 3 8))  
>>> '(5 4 9)  
(mapear (lambda (x) (+ x x)) '(3 5 3 1))  
>>> '(6 10 6 2)  
(filtrar even? '(2 3 4 6 7 10))  
>>> '(2 4 6 10)
```